HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI

Multi-Agent Systems & Collaborative Reasoning

Khảo sát, phân tích và đánh giá 8 nghiên cứu năm 2026

Nhóm thực hiện

Trần Xuân Hương
Vũ Tùng Dương
Phạm Việt Dũng
Lê Văn Đức

Giảng viên hướng dẫn

PGS.TS. Lê Thanh Hương

Học phần

IT4772 – Xử lý ngôn ngữ tự nhiên
Báo cáo Bài tập lớn

ONE LOVE. ONE FUTURE.

Nội dung trình bày

1. Giới thiệu
2. Safety & Robustness
3. Orchestration & Control
4. Communication & Interaction
5. Domain Applications
6. So sánh & kết luận

01

Giới thiệu

LLM-based MAS: nhiều agent cùng suy luận

Ý tưởng

- Mỗi agent = backbone LLM + role prompt + (tùy chọn) tool.
- Các agent trao đổi message để cùng giải bài toán: planner, coder, critic, verifier…
- Framework tiêu biểu: CAMEL, AutoGen, MetaGPT, multi-agent debate.

Câu hỏi nghiên cứu năm 2026

Không còn là "cộng tác có giúp không?" mà là: hiệu quả chi phí · luồng thông tin · an toàn · ứng dụng chuyên ngành.

MAS = communication graph

Node là agent; cạnh $v_j \to v_i$: output của v_j làm context cho v_i.

$$r_i = \mathrm{LLM}(p_i,\, q,\, O_i)$$

Topology có thể tối ưu (GPTSwarm, G-Designer) hoặc cắt tỉa (AgentPrune).

Vì sao topology quan trọng?

Vừa là đòn bẩy hiệu năng, vừa là kênh lan truyền lỗi, vừa là bề mặt tấn công.

Bốn thách thức chính của MAS

C1 – Chi phí

Nhiều lần gọi LLM, context dài → token, năng lượng GPU, latency tăng nhanh.
⇒ ProgRouter, DAIN

C2 – Quá tải context

"Lost in the middle"; compaction hay pruning đều làm mất thông tin.
⇒ AGENT-RADAR

C3 – Lan truyền lỗi

Hallucination hoặc misinformation lan theo cạnh và bị khuếch đại.
⇒ HalluProp, TIDE

C4 – Bảo mật topology

Topology là tài sản trí tuệ; lộ topology → tấn công có chủ đích.
⇒ MIRAGE

Tám paper được khảo sát

#	Paper	Chủ đề chính	Nộp	Nhóm
1	MIRAGE	Che giấu topology bằng phantom structure	29/09	Safety
2	MAC-RRG	Multi-agent cho radiology report generation	20/09	Application
3	ProgRouter	Routing LLM theo tiến độ, có ngân sách	26/08	Orchestration
4	TIDE	Misinformation khi sự thật bị phân tán	04/08	Safety
5	HalluProp	Dự báo rủi ro hallucination pre-hoc	29/07	Orchestration
6	UrbanAgent	Urban region profiling (KDD'26)	15/07	Application
7	DAIN	Dynamic agent interaction network	29/06	Communication
8	AGENT-RADAR	Attention steering cho MAS	28/05	Communication

Cách chọn bài

- Nguồn: arXiv.
- Abstract chứa "multi-agent" và "collaborative reasoning".
- 8 bài mới nhất (05–09/2026).

Lưu ý: 7/8 là preprint; chỉ UrbanAgent đã được nhận ở KDD 2026.

Taxonomy: bốn hướng nghiên cứu

Safety & Robustness

Hệ thống có chống chịu được tấn công và thông tin sai?
#1 MIRAGE · #4 TIDE

Orchestration & Control

Ai chạy, chạy bằng model nào, agent nào rủi ro?
#3 ProgRouter · #5 HalluProp

Communication & Interaction

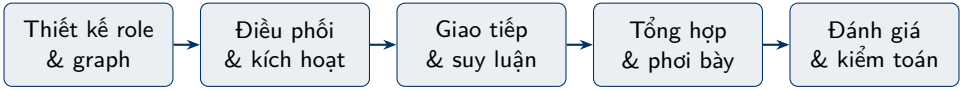
Agent trao đổi gì, với ai, mức độ nào?
#7 DAIN · #8 AGENT-RADAR

Domain Applications

Áp dụng cộng tác vào bài toán chuyên ngành.
#2 MAC-RRG · #6 UrbanAgent

Phân loại theo mục tiêu chính; nhiều bài thuộc nhiều nhóm (VD: HalluProp vừa là orchestration vừa là safety).

Mỗi phương pháp can thiệp ở đâu trong lifecycle của MAS



Thiết kế role & graph	Điều phối & kích hoạt	Giao tiếp & suy luận	Tổng hợp & phơi bày	Đánh giá & kiểm toán
HalluProp (pre-hoc)	ProgRouter DAIN (gating)	AGENT-RADAR DAIN (graph) UrbanAgent MAC-RRG	MIRAGE	TIDE

02

Safety & Robustness

MIRAGE: che giấu topology bằng phantom structure

Bài toán: tấn công CIA suy ra topology chỉ qua truy vấn hộp đen; lộ topology → mất tài sản trí tuệ, lộ agent then chốt.

Phương pháp: 3 bước

1. *Phantom topology:* đi dây lại cạnh, độ lệch $\rho \geq \rho_0$ (k-anonymity).
2. *Semantic edge realization:* LLM sinh bằng chứng giả cho phantom edge; paraphrase để giấu cạnh thật.
3. *Dual-view execution:* chạy trên G thật, chỉ phơi bày G'.

AUC / ACC / F1 của tấn công (G-Designer, thấp hơn là tốt)

Dataset	Delimiters	MIRAGE
MMLU	0.76 / 0.61 / 0.64	**0.67 / 0.50 / 0.49**
GSM8K	0.80 / 0.76 / 0.67	**0.64 / 0.51 / 0.42**
SVAMP	0.70 / 0.68 / 0.71	**0.64 / 0.48 / 0.39**
HumanEval	0.76 / 0.65 / 0.56	**0.68 / 0.44 / 0.52**

Hạn chế

Chỉ 1 tấn công, không có adaptive attacker; AUC vẫn 0.64–0.68 (lý tưởng 0.5); không báo cáo chi phí gọi LLM thêm.

TIDE: misinformation làm chệch collective fact recovery

Thiết lập

- 120 môi trường Hi-ToM; 5 agent, mỗi agent chỉ thấy một phần chuỗi vị trí.
- C0: tất cả trung thực · C1: key agent nói dối (Event / Coverage / Source).
- GPT-5.5, DeepSeek-v4-pro, Grok-4.5; đo cả evidence lineage.

Kết quả chính

- Tỉ lệ đa số đúng: **72.50%** → **14.17%** (C0 → C1).
- Testimony sai được chấp nhận **70.7%**, so với **25.4%** của testimony đúng.
- Propagation order: **2.66** (sai) so với **1.33** (đúng).

Thông điệp

Thảo luận *khuếch đại* chiều hướng của bằng chứng ban đầu thay vì sửa sai; ∼85% khoảng cách hình thành ngay khi chia sẻ testimony. Observer giảm 90% decoy majority nhưng không tăng đáp án đúng. Hạn chế: 1 tác vụ hẹp, chưa có defense.

03

Orchestration & Control

ProgRouter: Đặt vấn đề

Bối cảnh

- Multi-agent LLM workflow giải tốt task phức tạp nhưng đắt: gọi LLM nhiều lần, context tích lũy dài $\rightarrow$ tốn token, năng lượng GPU, latency.
- Cascade routing (RouteLLM, CASCADIA, MasRouter) chọn model *một lần cho cả query* – không theo kịp trạng thái thay đổi qua từng bước.

Câu hỏi của paper

Chọn LLM cho **từng bước** của workflow thế nào để giữ chất lượng mà vẫn nằm trong ngân sách chi phí?

ProgRouter: Problem setting

Workflow $w = \{C, R, M\}$

- C: coordinator agent – chia subtask, giữ *ledger* trạng thái s_t, chọn role r_t cho bước tiếp theo.
- R: các role worker (analyst, coder…).
- M: model zoo nhiều cỡ; $M_r \subseteq M$ là các model dùng được cho role r.

Bài toán tối ưu online trên dòng W task: tìm policy $\pi(s_t, r_t) \mapsto m_t \in M_{r_t}$

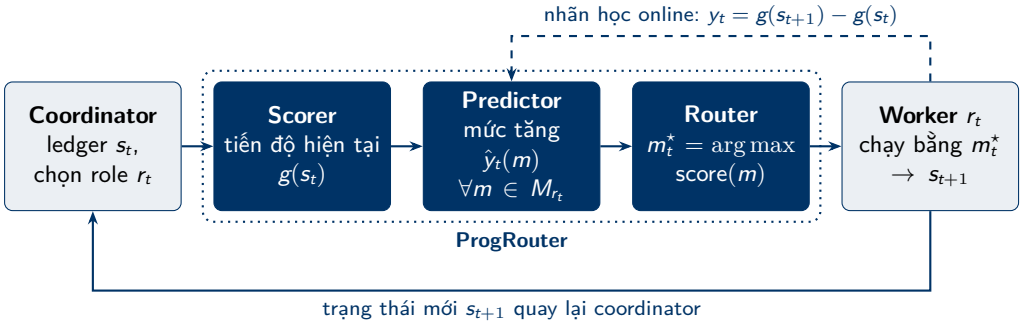
$$\max_{\pi} \ \frac{1}{W}\sum_w P(w)$$
$$\text{s.t. } \sum_t E(m_t) \le E_w, \quad \sum_t \Gamma(m_t) \le \Gamma_w \quad \forall w$$
$$\frac{1}{W}\sum_w \sum_t E(m_t) \le \tilde{E}$$

$P(w) \in [0, 1]$: chất lượng cuối; E, Γ: năng lượng, thời gian; $\tilde{E}$: ngân sách năng lượng trung bình dài hạn.

Hai khó khăn

(1) $P(w)$ chỉ biết khi workflow kết thúc → không chấm được từng bước.
(2) Ràng buộc trung bình phụ thuộc các task tương lai chưa biết.

ProgRouter: Tổng quan phương pháp



Ở mỗi bước, coordinator chọn role; ProgRouter **đo** tiến độ hiện tại, **dự đoán** mỗi model ứng viên giúp tiến thêm bao nhiêu, rồi **chọn** model cân bằng giữa tiến độ và ngân sách.

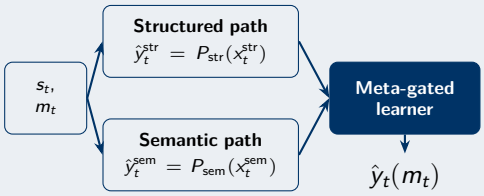
ProgRouter: Đo và dự đoán tiến độ

Multi-View Task Progress Representation

$$g(s_t) = b + \alpha\, r + \beta\, d + \gamma\, e \;\in [0, 1]$$

- b: **Overall outcome** – Trạng thái tổng quát của workflow.
- r: **Subtask completion** – Tỉ lệ subtask hoàn thành.
- d: **Progress trend** – Tiến độ của workflow.
- e: **State quality** – Mức ý nghĩa về ngữ nghĩa và cấu trúc của lần chuyển trạng thái gần nhất.

Online Task Progress Predictor



$$\hat{y}_t = P_{\text{meta}}\left(\hat{y}_t^{\text{str}},\ \hat{y}_t^{\text{sem}}\right)$$

ProgRouter: Chọn model ở mỗi bước

Ở bước t, chọn $m_t^\star = \arg\max_{m_t \in M_{r_t}} \text{score}(m_t)$, với mỗi model ứng viên m_t:

$$\text{score}(m_t) = \underbrace{V(1-g(s_t))\,\hat{y}_t(m_t)}_{\text{lợi ích}} - \underbrace{Q_w\big(E(m_t)-\tilde{E}\big)}_{\text{phạt dài hạn}} - \underbrace{c_t^{\Gamma}\,\Gamma(m_t) - c_t^{E}\,E(m_t)}_{\text{phạt trong task}}$$

Lợi ích: task còn xa đích (g nhỏ) thì tiến độ đáng giá hơn → dám gọi model mạnh.

Phạt dài hạn

Hàng đợi ảo cộng dồn phần vượt ngân sách sau mỗi task:

$$Q_{w+1} = \max\big\{0,\ Q_w + \textstyle\sum_{t=1}^{T_w} E(m_t) - \tilde{E}\big\}$$

Phạt trong task

Tăng dần khi task gần hết ngân sách (c_t^{Γ} tương tự, theo thời gian):

$$c_t^{E} = \exp\Big(\frac{E(m_t)+\sum_{i<t} E(m_i)}{E_w}\Big)$$

ProgRouter: Thực nghiệm và kết quả chính

Thiết lập: 4 benchmark (code, toán, RAG QA); model zoo 7–9 LLM từ 0.5B đến 35B; coordinator Qwen3-Coder-Next 80B; năng lượng GPU đo thật bằng NVML.

Pass/Precision % / năng lượng J (ngân sách $\tilde{E}$)	HumanEval+ (4,800)	MBPP (4,500)	MATH-500 (7,000)	ASQA (19,000)
Model lớn nhất, dùng một mình	97.1 / 5,443	93.9 / 4,804	92.2 / 26,276	92.3 / 21,887
Educated Guessing	91.5 / 4,916	65.2 / 3,831	79.3 / 7,023	90.8 / 23,423
CASCADIA	84.8 / 4,658	78.5 / 3,857	**87.8** / 6,875	89.8 / 27,857
MasRouter	90.9 / 4,483	67.8 / 3,700	73.6 / 8,294	89.8 / 16,368
ProgRouter	**93.0** / 4,796	**79.4** / **3,376**	84.3 / **6,112**	**92.1** / 18,373

Đỏ: vượt ngân sách năng lượng dài hạn; **đậm**: tốt nhất trong các phương pháp không vượt ngân sách.

- Dùng một model cố định đều thua: model lớn vượt ngân sách; model nhỏ cũng tốn vì phải sửa lỗi nhiều lần.
- ProgRouter luôn trong ngân sách, tốt nhất ở 3/4 benchmark; phần lớn lời gọi dồn vào model nhỏ (84.3% Qwen2.5-Coder 0.5B trên HumanEval+, 91.0% Granite 3B trên MATH-500).

HalluProp: Đặt vấn đề

Bối cảnh

- Trong MAS, hallucination lan truyền: một agent sai, các agent khác chấp nhận, lặp lại và khuếch đại → cả hệ thống ra kết quả sai.
- Các phương pháp hiện có (SelfCheckGPT, LLM Debate, Inspector, GUARDIAN) đều **post-hoc**: chạy hết MAS rồi mới phát hiện – lỗi đã lan, tốn 53–308 s/query.

Câu hỏi của paper

Có thể dự đoán agent nào sẽ hỏng **trước khi** các agent bắt đầu giao tiếp?

HalluProp: Problem setting

MAS là đồ thị có hướng $M = (V, E)$

- $V = \{v_1, \ldots, v_N\}$: N agent; agent v_i có system prompt (role) p_i.
- $E \in \{0,1\}^{N \times N}$: cạnh $v_j \to v_i$ nghĩa là v_i đọc output của v_j.
- q: câu hỏi của người dùng.
- $H_i \in \{0,1\}$: agent v_i có hallucinate không (thực nghiệm: trả lời sai so với đáp án).

Mục tiêu: ước lượng rủi ro của từng agent

$$h_i = P\big(H_i = 1 \mid \{p_i\}_{i=1}^{N},\ q,\ E\big)$$

Đầu vào **chỉ có** role, câu hỏi và topology – chưa có câu trả lời nào.

Tách rủi ro làm hai phần

- **Intrinsic** $\pi_i \in [0,1]$: rủi ro agent v_i tự sai.
- **Propagation** $t_{j \to i} \in [0,1]$: xác suất lỗi của v_j lây sang v_i.

Gộp π và $T = [t_{j \to i}]$ trên đồ thị để ra $h = [h_1, \ldots, h_N]$.

HalluProp: Intrinsic risk – role có hợp với câu hỏi không?

1. Encode role và câu hỏi vào cùng không gian:

$$e_i^R = \mathrm{Enc}(p_i), \qquad e^Q = \mathrm{Enc}(q)$$

2. Gate theo câu hỏi – chọn các chiều của role liên quan đến câu hỏi:

$$g_i = \sigma\big(W_{\text{gate}}\,(e_i^R \odot e^Q) + b_g\big)$$

3. Tách role làm hai phần:

$$\underbrace{\kappa_i = e_i^R \odot g_i}_{\text{competence}}, \qquad \underbrace{\rho_i = \mathrm{LN}(e_i^R - \kappa_i)}_{\text{susceptibility}}$$

4. Rủi ro:

$$\pi_i = \sigma\big(w_\rho^\top \rho_i - w_\kappa^\top \kappa_i + b_\pi\big)$$

Trực giác

- κ_i (*competence*): phần của role khớp với câu hỏi.
- ρ_i (*susceptibility*): phần còn lại – lạc đề, mơ hồ hoặc gây nhiễu.
- Nhiều nhiễu, ít năng lực $\rightarrow \pi_i$ cao.

Chỉ so độ tương đồng tổng thể giữa role và câu hỏi là không đủ, vì role prompt thường mơ hồ hoặc chứa nội dung gây nhiễu.

HalluProp: Lan truyền và Noisy-OR

Propagation qua cạnh $v_j \to v_i$

$$t_{j\to i} = \sigma\big(\gamma_{j,i}\, \kappa_j^{\top} W_{\text{sem}}\, \rho_i + b_t\big)$$

- κ_j: bên gửi có sức ảnh hưởng.
- ρ_i: bên nhận dễ bị lây.
- $\gamma_{j,i} = \phi(\hat{k}_{\text{out}}(j), \hat{k}_{\text{in}}(i)) \geq 1$: khuếch đại theo bậc ra / bậc vào.
- $t_{j\to i} = 0$ nếu không có cạnh.

Noisy-OR trên đồ thị

$$h_i = 1 - (1 - \pi_i) \prod_{j \in N_{\text{in}}(i)} \big(1 - t_{j\to i}\, h_j\big)$$

Agent *không* hỏng $\iff$ tự không sai **và** không bị lây từ agent nào.

Huấn luyện 2 giai đoạn: học π với nhãn từng agent *trước* giao tiếp, rồi học T với nhãn *sau* giao tiếp, BCE trên h.

HalluProp: Thực nghiệm và kết quả

Thiết lập

- MMLU, MATH, MedQA (500 câu mỗi bộ).
- 4 backbone: GPT-3.5, GPT-4o, Llama-3-8B, Qwen-2-7B.
- Topology ngẫu nhiên; role sạch lẫn role bị nhiễu; 3 vòng giao tiếp.

Phát hiện agent hỏng

- AUROC trung bình **84.6**; Hit@1 **83.9**; Recall@3 **76.2**.

Tốc độ

- Dưới **1 s**, so với 53–308 s của post-hoc (nhanh hơn $> 65\times$).

Accuracy sau can thiệp vào agent rủi ro nhất (trung bình 4 LLM $\times$ 3 dataset)

Phương pháp	Accuracy %
LLM Debate	55.7
Inspector	54.4
SelfCheckGPT	56.7
GUARDIAN	63.5
HalluProp-Pruning	63.6
HalluProp-Refinement	**67.6**

Pruning: cắt mọi cạnh ra của agent đó. *Refinement*: đổi role của nó thành verifier thận trọng. Các baseline đều post-hoc, phải chạy hết MAS.

HalluProp: Ablation

AUROC trung bình trên GPT-3.5-turbo (3 dataset)

Biến thể	AUROC	Giảm
HalluProp đầy đủ	**84.8**	–
bỏ intrinsic risk	61.2	-23.6
bỏ propagation	66.9	-17.9
bỏ topology (γ)	72.1	-12.7

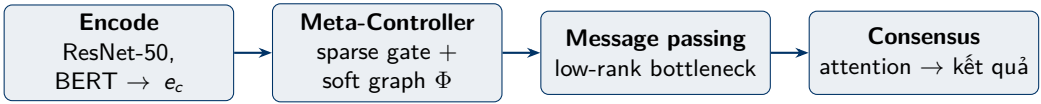
- **Bỏ intrinsic risk** (thay bằng độ tương đồng role–câu hỏi): giảm mạnh nhất $\rightarrow$ tách competence / susceptibility là cần thiết.
- **Bỏ propagation** ($t_{j \to i} = 0$, khi đó $h_i = \pi_i$): lỗi trong MAS không chỉ là lỗi riêng lẻ mà còn lan qua giao tiếp.
- **Bỏ topology**: không nhận ra các agent phát tán mạnh.

Cả ba thành phần đều cần thiết; thiếu bất kỳ thành phần nào, AUROC giảm từ 12.7 đến 23.6 điểm.

04

Communication & Interaction

DAIN: Dynamic Agent-based Interaction Network



- 8 agent theo PID: 3 synergy · 3 uniqueness · 2 redundancy.
- Loss phạt kích hoạt và mật độ giao tiếp: $\lambda\|g\|_1 + \lambda\|\Phi\|^2$.
- Chỉ **3.2–5.5 / 8** agent được kích hoạt.
- ADNI accuracy **73.5** so với 70.9 (best baseline).

Lưu ý khi đánh giá

- "Agent" là neural module, không phải LLM: bản chất là MoE có message passing.
- Bỏ communication: -1.4; bỏ dynamic scheduling: -1.7 điểm (ADNI).
- Số liệu không nhất quán giữa các bảng; baseline yếu.

AGENT-RADAR: attention steering theo context relevance

Ý tưởng

- Giữ toàn bộ lịch sử, không tóm tắt hay cắt bớt.
- Chấm điểm từng câu; câu có điểm $\geq \theta = 0.65$ được nhấn mạnh bằng SPA.
- Training-free, cắm được vào GPTSwarm, AutoGen, debate.

Điểm của câu s_k

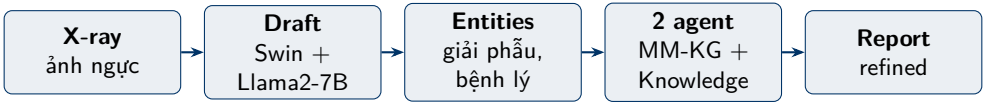
$$\underbrace{\lambda_s^{d_G-1}}_{\text{spatial decay}} \times \underbrace{\lambda_t^{t-\tau-1}}_{\text{temporal decay}} \times \underbrace{\cos(s_k, q)}_{\text{semantic relevance}}$$

Hạn chế: latency +23% · cần white-box (logit) → không dùng với API model · tune trên MuSiQue (cũng là test set).

05

Domain Applications

MAC-RRG: multi-agent cho radiology report generation



Kết quả

- BLEU-4: **0.197 / 0.128 / 0.105** trên IU X-Ray / MIMIC-CXR / CheXpert Plus.
- Clinical F1 (CheXpert Plus): **0.295** so với 0.260 của R2GenGPT.

Đánh giá phản biện

- "Agent" = module retrieval tất định, không giao tiếp hay thương lượng.
- "Iterative" thực tế chỉ 1 lượt tinh chỉnh.
- BLEU tăng nhỏ (0.101 $\rightarrow$ 0.105); không kiểm định thống kê, không có bác sĩ đánh giá.

UrbanAgent: collaborative reasoning cho urban region profiling

Stage A · Tool-augmented agents

- 4 agent theo modality: satellite, text, POI, 3D.
- Qwen3-VL-4B + 4 tool (web search, crop-zoom, nightlight, ảnh lịch sử).
- SFT → GRPO, reward = outcome + format + process.

Stage B · Relation-aware graph

- *Corroboration:* dự đoán gần nhau → củng cố.
- *Rectification:* lệch nhiều → sửa.
- *Supervisory:* agent tự tin hơn hướng dẫn; 5 vòng GRU → final judge.

Kết quả: R^2 **0.63–0.65** trong miền (carbon, dân số, GDP); ≈ **0.60** trên thành phố chưa thấy; **+8.1%** so với baseline mạnh nhất.

Đánh giá phản biện

"Debate" là GNN/GRU được huấn luyện, không phải tranh luận bằng ngôn ngữ. Dữ liệu nhỏ (2,000 vị trí); text sinh từ chính ảnh vệ tinh → các modality không độc lập.

06

So sánh & kết luận

So sánh hệ thống tám paper

Paper	Loại đóng góp	Training	White-box	Chi phí	Thống kê
MIRAGE	Defense	✗	✗	✗	✗
TIDE	Benchmark	✗	✗	✗	✓
ProgRouter	Router	online	✗	✓ năng lượng	✗
HalluProp	Risk predictor	✓	✗	∼ thời gian	✗
DAIN	Kiến trúc fusion	✓	–	∼ tham số	✗
AGENT-RADAR	Plug-in	✗	✓	∼ thời gian	✗
MAC-RRG	Ứng dụng	✓	–	✗	✗
UrbanAgent	Ứng dụng	✓	–	✗	✗

Đọc bảng: chỉ TIDE có kiểm định thống kê; chỉ ProgRouter đo năng lượng GPU thực; chỉ AGENT-RADAR bắt buộc white-box. Cùng benchmark nhưng khác tập con, topology, backbone → không so trực tiếp được con số.

Ba nhận định xuyên suốt

1. **Communication graph là trung tâm.**
   6/8 bài thao tác trực tiếp: giấu (MIRAGE), phân tích (HalluProp), đo khoảng cách (RADAR), học (DAIN, UrbanAgent), truy vết (TIDE).
2. **Lan truyền là rủi ro cốt lõi.**
   Thảo luận khuếch đại sai lệch thay vì sửa sai (TIDE); hallucination lan theo cạnh (HalluProp).
3. **Khái niệm "agent" bị nới rộng.**
   LLM agent thật (5 bài) · LLM + aggregator học được (UrbanAgent) · neural / retrieval module (DAIN, MAC-RRG).

Hiệu quả không đến từ nhiều agent hơn: ProgRouter dùng chủ yếu model 0.5B; DAIN tốt nhất với $\tau = 4/8$ agent.

Research gap và hướng phát triển

Research gap

- G1 Chưa có benchmark chung: chất lượng + chi phí + robustness.
- G2 7/8 bài không có phương sai hay kiểm định thống kê.
- G3 Quy mô nhỏ: ≤ 8 agent, 2–5 vòng, tác vụ ngắn.
- G4 Chưa có adaptive attacker; chưa có cách khôi phục sự thật.
- G5 Phương pháp tối ưu chưa được thử trong điều kiện đối kháng.

Hướng phát triển

- *Benchmark 3 trục:* chất lượng · chi phí · robustness (giao thức C0/C1 của TIDE).
- *Provenance-aware communication:* $\mathrm{score}(\mathrm{RADAR}) \times (1 - \hat{h}_j) \times \mu^{\mathrm{hop}}$.
- *Risk-aware routing:* ProgRouter + HalluProp, bước rủi ro $\rightarrow$ model mạnh.
- *Verifier agent chủ động:* đòi bằng chứng trực tiếp trước khi đồng thuận.

Kết luận

1. **Communication graph là trung tâm** – quyết định hiệu năng, lan truyền rủi ro, cần được bảo vệ.
2. **Lan truyền là rủi ro cốt lõi** – thảo luận khuếch đại sai lệch thay vì sửa sai.
3. **Hiệu quả = kích hoạt đúng** – đúng agent, đúng model, chú ý đúng nội dung.
4. **Thực nghiệm còn yếu** – quy mô nhỏ, thiếu kiểm định, thiếu đo chi phí.

Câu hỏi của lĩnh vực đang chuyển

Từ *"nhiều agent có tốt hơn một agent?"*
sang **"làm sao để MAS hiệu quả, đáng tin cậy và an toàn?"**

Phân công

Thành viên	Paper phụ trách
Trần Xuân Hương	MIRAGE · TIDE
Vũ Tùng Dương	ProgRouter · HalluProp
Phạm Việt Dũng	DAIN · AGENT-RADAR
Lê Văn Đức	MAC-RRG · UrbanAgent

HUST

hust.edu.vn fb.com/dhbkhn

CẢM ƠN ĐÃ LẮNG NGHE!

Q&A